

Step 2 — Parallel Lookups With `RunnableParallel` — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

All examples below are run from inside `project_13_langchain_patterns/`, continuing from Step 1's `router.py` and `handlers.py`.

Basic Version

Approach 1 — sequential lookups, no `RunnableParallel` yet

```
# lookups.py
import time

def get_account_info(inputs):
    # stand-in for a real database or CRM API call
    time.sleep(1)
    return {"plan": "Pro", "customer_since": "2022-01-01"}

def get_order_history(inputs):
    # stand-in for a real orders-service API call
    time.sleep(1)
    return {"recent_orders": ["INV-2201", "INV-2150"]}

import time as timer

ticket_input = {"ticket": "I was charged twice."}

start = timer.perf_counter()
account = get_account_info(ticket_input)
orders = get_order_history(ticket_input)
elapsed = timer.perf_counter() - start

print("Sequential:", account, orders)
print(f"Sequential took {elapsed:.2f}s")
```

Expected output:

```
Sequential: {'plan': 'Pro', 'customer_since': '2022-01-01'} {'recent_orders': ['I  
Sequential took 2.00s
```

This works, but notice the time: about 2 seconds, because `get_order_history` doesn't even start until `get_account_info` has completely finished, even though neither one needed the other's answer.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — the same two lookups, through `RunnableParallel`

```
# research.py
from langchain_core.runnables import RunnableParallel, RunnableLambda, RunnablePassthrough
from lookups import get_account_info, get_order_history

def build_research_chain():
    return RunnableParallel(
        account_info=RunnableLambda(get_account_info),
        order_history=RunnableLambda(get_order_history),
        ticket=RunnablePassthrough(),
    )
```

```
# main.py
import time
from research import build_research_chain

research_chain = build_research_chain()
ticket_input = {"ticket": "I was charged twice."}

start = time.perf_counter()
result = research_chain.invoke(ticket_input)
elapsed = time.perf_counter() - start

print("Parallel result:", result)
print(f"Parallel took {elapsed:.2f}s")
```

Expected output:

```
Parallel result: {'account_info': {'plan': 'Pro', 'customer_since': '2022-01-01'}}
Parallel took 1.01s
```

Same two lookups, same 1-second sleep each, but the total time is close to 1 second instead of 2 — `RunnableParallel` ran both functions in separate threads at the same time.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Approach 2 — wiring it into `handlers.py` 's billing and technical chains

```
# handlers.py (updated)
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_openai import ChatOpenAI
from research import build_research_chain

model = ChatOpenAI(model="gpt-4o-mini", temperature=0.3)
research_chain = build_research_chain()

billing_prompt = ChatPromptTemplate.from_template(
    "You are a billing support agent. The customer's plan is {account_info} "
    "and their recent orders are {order_history}. "
    "Ticket: {ticket}"
)
technical_prompt = ChatPromptTemplate.from_template(
    "You are a technical support agent. The customer's plan is {account_info} "
    "and their recent orders are {order_history}. "
    "Ask a clarifying troubleshooting question if needed. Ticket: {ticket}"
)
general_prompt = ChatPromptTemplate.from_template(
    "You are a friendly general support agent. Ticket: {ticket}"
)

billing_chain = research_chain | billing_prompt | model | StrOutputParser()
technical_chain = research_chain | technical_prompt | model | StrOutputParser()
general_chain = general_prompt | model | StrOutputParser() # no lookups needed
```

`general_chain` deliberately skips `research_chain` entirely — a referral-program question never needs account or order data, so running those lookups for it would just be wasted latency and a wasted API call.

Difference from Basic: Basic proves the timing gap exists with two plain function calls, no LangChain involved yet. Intermediate wires the same two functions into a real `RunnableParallel`, merges in the original ticket with `RunnablePassthrough`, and only attaches the research step to the two handlers that actually need it.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — the honest side-by-side proof

```
# test_pipeline.py (excerpt) -- prove it, don't just claim it
import time
from lookups import get_account_info, get_order_history
from research import build_research_chain

ticket_input = {"ticket": "I was charged twice."}

# Sequential
start = time.perf_counter()
get_account_info(ticket_input)
get_order_history(ticket_input)
sequential_time = time.perf_counter() - start

# Parallel
research_chain = build_research_chain()
start = time.perf_counter()
research_chain.invoke(ticket_input)
parallel_time = time.perf_counter() - start

print(f"Sequential: {sequential_time:.2f}s")
print(f"Parallel:    {parallel_time:.2f}s")

assert parallel_time < sequential_time * 0.7, (
    "Parallel run wasn't meaningfully faster -- check both lookups are "
    "actually wrapped in RunnableLambda and passed into RunnableParallel, "
    "not called directly somewhere else first."
)
print("Confirmed: RunnableParallel is really running these concurrently.")
```

Expected output:

```
Sequential: 2.00s
Parallel:   1.01s
Confirmed: RunnableParallel is really running these concurrently.
```

Approach 2 — what happens if a real, more expensive lookup calls a shared, limited resource

```
# a note worth writing down, not new code:
# RunnableParallel's concurrency is genuinely useful here because the two
# lookups are independent -- but if get_account_info and get_order_history
# both hit the *same* rate-limited API, running them "in parallel" doesn't
# save time at all; it just makes both calls compete for the same limit at
# the same moment. RunnableParallel merges results at one fixed point --
# it has no idea whether the things it's running concurrently would
# actually benefit from that, or would just collide. That judgment call
# is still yours to make before reaching for RunnableParallel, not
# something the tool checks for you.
```

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate wires up the real `RunnableParallel` call and shows one honest before/after number. Approach 1 turns that into a real, automated assertion your test suite can rerun and trust, instead of a one-off print statement you eyeballed once. Approach 2 isn't code at all — it's the limit of what `RunnableParallel` can tell you: it will happily run two things "at once" even when doing so provides no real benefit (or actively hurts, against a shared rate limit), because it has no idea what's actually behind each function it's calling.

Which one should you actually write? Approach 1's assertion, in your real test suite — a timing claim that isn't backed by a real, rerunnable check is just a comment someone will eventually stop trusting. Before reaching for `RunnableParallel` on a new pair of lookups in real work, also do Approach 2's sanity check by hand: are these two calls actually independent, and do they hit different enough resources that running them at once is a genuine win, not just a more confusing way to write two sequential calls?